

Министерство образования и науки Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Санкт-Петербургский политехнический университет Петра Великого»

Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта
02.03.01 Математика и компьютерные науки

Отчёт по дисциплине
Вычислительная математика
Лабораторная работа №2
«Аппроксимация»
Вариант 18

Выполнил:
студент группы 5130201/40003
Джабраилов А.В.
Подпись: _____

Принял:
К. физ.-мат. наук, доц., доц.
Пак В.Г.
Подпись: _____
«_____» _____ 2026 г.

Санкт-Петербург, 2026

1 Задание

Для данной табличной функции построить многочлены с наилучшим среднеквадратичным отклонением первой и второй степени методом наименьших квадратов.

2 Цель работы

Научиться аппроксимировать табличные функции полиномами методом наименьших квадратов.

3 Ход работы

Таблица 1: Таблица значений (часть 1).

x_i	0,1	0,3	0,5	0,7	0,9	1,1	1,3	1,5	1,7	1,9
y_i	-0,86	-0,77	-0,56	-0,46	-0,28	-0,24	-0,36	-0,43	-0,56	-0,59

Таблица 2: Таблица значений (часть 2).

x_i	2,1	2,3	2,5	2,7	2,9	3,1	3,3	3,5	3,7	3,9
y_i	-0,70	-1,01	-1,03	-1,47	-1,68	-1,93	-2,28	-2,53	-2,93	-3,07

Алгоритм вычислений следующий:

1. Построение матрицы Вандермонда, т.к. т.о. можно пробразовать задачу подбора коэффициентов многочлена в задачу решения СЛУ: умножением матрицы A на вектор неизвестных коэффициентов a получаем вектор значений многочлена в точках x ;
2. Вычисление коэффициентов a методом Гаусса;
3. Вычисление среднеквадратичного отклонения.

Для выполнения работы была написана программа на языке C++. Программа имеет модульную структуру и состоит из четырёх функций:

1. *solveLinearSystem* — решает систему линейных уравнений $Ax = b$.

2. *polyval* — вычисляет значение полинома.
3. *approximation* — реализует метод наименьших квадратов.
4. *main* — точка входа программы.

Для вычислений используются библиотеки *vector* и *cmath*.

Текст исходного кода функций приведён в приложении.

4 Результаты

1. Многочлен первой степени: $P_1(x) = 0.13 - 0.66x$. Среднее квадратичное отклонение: 0,45.
2. Многочлен второй степени: $P_2(x) = -0.87 + 0.84x - 0.38x^2$. Среднее квадратичное отклонение: 0,10.

5 Приложение

Listing 1: Файл Lab2.cpp

```
1 #include <iostream>
2 #include <vector>
3 #include <cmath>
4 #include <iomanip>
5
6 // Solve linear system Ax = b using Gaussian elimination with
7 // partial pivoting
8 std::vector<double> solveLinearSystem(std::vector<std::vector<
9 double>> A, std::vector<double> b) {
10     int n = b.size();
11
12     // Forward elimination with partial pivoting
13     for (int k = 0; k < n; k++) {
14         // Find pivot
15         int maxRow = k;
16         for (int i = k + 1; i < n; i++) {
17             if (std::abs(A[i][k]) > std::abs(A[maxRow][k])) {
18                 maxRow = i;
19             }
20         }
21         // Swap rows k and maxRow
22         std::swap(A[k], A[maxRow]);
23         std::swap(b[k], b[maxRow]);
24
25         // Eliminate below
26         for (int i = k + 1; i < n; i++) {
27             double factor = A[i][k] / A[k][k];
28             for (int j = k + 1; j < n; j++) {
29                 A[i][j] -= factor * A[k][j];
30             }
31             b[i] -= factor * b[k];
32         }
33     }
34
35     // Back substitution
36     std::vector<double> x(n, 0);
37     x[n - 1] = b[n - 1] / A[n - 1][n - 1];
38     for (int k = n - 2; k >= 0; k--) {
39         double sum = b[k];
40         for (int j = k + 1; j < n; j++) {
41             sum -= A[k][j] * x[j];
42         }
43         x[k] = sum / A[k][k];
44     }
45     return x;
46 }
```

```

17         }
18     }
19
20     // Swap rows
21     std::swap(A[k], A[maxRow]);
22     std::swap(b[k], b[maxRow]);
23
24     // Eliminate column
25     for (int i = k + 1; i < n; i++) {
26         double factor = A[i][k] / A[k][k];
27         for (int j = k; j < n; j++) {
28             A[i][j] -= factor * A[k][j];
29         }
30         b[i] -= factor * b[k];
31     }
32 }
33
34 // Back substitution
35 std::vector<double> x(n);
36 for (int i = n - 1; i >= 0; i--) {
37     x[i] = b[i];
38     for (int j = i + 1; j < n; j++) {
39         x[i] -= A[i][j] * x[j];
40     }
41     x[i] /= A[i][i];
42 }
43
44 return x;
45 }
46
47 // Evaluate polynomial at point x
48 double polyval(const std::vector<double>& coeffs, double x) {
49     double result = 0.0;
50     for (int i = coeffs.size() - 1; i >= 0; i--) {
51         result = result * x + coeffs[i];
52     }
53     return result;
54 }
55
56 // Polynomial approximation using least squares
57 std::pair<std::vector<double>, double> approximation(

```

```

58 const std::vector<double>& x,
59 const std::vector<double>& y,
60 int degree) {
61
62     int n = x.size();
63     int m = degree + 1;
64
65     // Build Vandermonde matrix (increasing powers)
66     std::vector<std::vector<double>> A(n, std::vector<double>
67 >(m));
68     for (int i = 0; i < n; i++) {
69         for (int j = 0; j < m; j++) {
70             A[i][j] = std::pow(x[i], j);
71         }
72     }
73
74     // Solve normal equations: (A^T * A) * coeffs = A^T * y
75     std::vector<std::vector<double>> AtA(m, std::vector<
76 double>(m, 0.0));
77     std::vector<double> Aty(m, 0.0);
78
79     for (int i = 0; i < m; i++) {
80         for (int j = 0; j < m; j++) {
81             for (int k = 0; k < n; k++) {
82                 AtA[i][j] += A[k][i] * A[k][j];
83             }
84             for (int k = 0; k < n; k++) {
85                 Aty[i] += A[k][i] * y[k];
86             }
87         }
88     }
89
90     // Solve for coefficients
91     std::vector<double> coeffs = solveLinearSystem(AtA, Aty);
92
93     // Calculate fitted values and MSE
94     double mse = 0.0;
95     for (int i = 0; i < n; i++) {
96         double y_fit = polyval(coeffs, x[i]);
97         mse += std::pow(y[i] - y_fit, 2);
98     }

```

```

97     mse /= n;
98
99     double rmse = std::sqrt(mse);
100
101     return {coeffs, rmse};
102 }
103
104 int main() {
105     std::vector<double> x = {
106         0.1, 0.3, 0.5, 0.7, 0.9, 1.1, 1.3, 1.5, 1.7, 1.9,
107         2.1, 2.3, 2.5, 2.7, 2.9, 3.1, 3.3, 3.5, 3.7, 3.9
108     };
109
110     std::vector<double> y = {
111         -0.86, -0.77, -0.56, -0.46, -0.28, -0.24, -0.36,
112         -0.43, -0.56, -0.59,
113         -0.70, -1.01, -1.03, -1.47, -1.68, -1.93, -2.28,
114         -2.53, -2.93, -3.07
115     };
116
117     // Perform approximations
118     auto [coeffs_1, rmse_1] = approximation(x, y, 1);
119     auto [coeffs_2, rmse_2] = approximation(x, y, 2);
120
121     // Output results
122     std::cout << std::fixed << std::setprecision(2);
123
124     std::cout << "\nFirst-degree polynomial (linear
125 approximation):" << std::endl;
126     std::cout << "P_1(x) = " << coeffs_1[0] << " + " <<
127 coeffs_1[1] << "*x" << std::endl;
128     std::cout << "Root mean square error: " << rmse_1 << std
129 ::endl;
130
131     std::cout << "\nSecond-degree polynomial (quadratic
132 approximation):" << std::endl;
133     std::cout << "P_2(x) = " << coeffs_2[0] << " + " <<
134 coeffs_2[1] << "*x + "
135 << coeffs_2[2] << "*x^2" << std::endl;
136     std::cout << "Root mean square error: " << rmse_2 << std
137 ::endl;

```

130

131

132

133

```
return 0;
```

```
}
```